EE366200 DSP Laboratory

Lab 5 Pixel Array Manipulation

111061220 林進成

---

1. Introduction

In this experiment, we explore fundamental geometric image transformations that form the foundation of digital image processing. The goal of Lab 5 is to understand how an image can be modified spatially by manipulating its pixel coordinates through mathematical operations such as flipping, rotation, shearing, and resizing. Instead of relying on high-level libraries like OpenCV or PIL, all implementations are carried out using pure NumPy operations, which provides a deeper understanding of how each pixel is mapped and interpolated during transformation. By directly controlling coordinate mappings and intensity resampling, we gain insight into how geometric modifications affect image structure and visual quality. This experiment also introduces the concept of forward and backward warping, two distinct approaches for defining coordinate correspondence between the input and output images. Through this lab, we not only implement these operations but also analyze their differences and practical implications in image manipulation and computer vision applications.

2. Objectives

The main objective of this laboratory exercise is to develop a clear understanding of geometric image transformations through hands-on implementation. By constructing each operation manually using NumPy, the experiment emphasizes the underlying mathematical principles that govern how images are spatially manipulated. Specifically, this lab aims to demonstrate how pixel coordinates can be mapped, resampled, and interpolated to achieve various transformation effects such as grayscale conversion, flipping, rotation, shearing, and resizing. Another important objective is to distinguish between forward and backward warping methods, recognizing their advantages and limitations in different scenarios. In addition, the lab seeks to strengthen programming skills in numerical computation, promote an analytical understanding of interpolation methods such as bilinear interpolation, and encourage careful observation of how each transformation influences both image geometry and visual appearance. Through these goals, the experiment provides a practical foundation for more advanced topics in computer vision and digital image analysis.

3. Approach

Two source images, denoted as Fig. 1 and Fig. 2, serve as the inputs for every operation so that the visual effects of each transformation can be compared on identical content.


Fig.1, Fig.2 Source image provided by TA(left) and my own choice(right)

Grayscale:

Grayscale conversion replaces each color triplet with its luminance computed by the BT.601 model, $Y = 0.299R + 0.587G + 0.114B$, and the single-channel result is replicated across three channels for consistency with subsequent stages. Applied to Fig. 1 and Fig. 2, the resulting achromatic images are shown as Fig. 3 and Fig. 4, where chromatic contrasts collapse to intensity contrast while geometric detail remains unchanged.


Fig.3, Fig.4 Grayscale of the image provided by TA(left) and my own choice(right)

Flipping:

Flipping is modeled as reflections about the image axes in the (u, v)-plane. A horizontal flip about the vertical axis is $(u, v) \mapsto (-u, v)$; a vertical flip about the horizontal axis is $(u, v) \mapsto (u, -v)$; composing both reflections yields a 180°rotation $(u, v) \mapsto (-u, -v)$. These are isometries that preserve distances and thus keep local sharpness and texture statistics intact while reversing global orientation. In the tested run, all these three flipping is applied to the inputs, and the resulting images are shown on Fig. 5 to Fig. 10.

Fig.5, Fig.6 Horizontal flip of the image provided by TA(left) and my own choice(right)

Fig.7, Fig.8 Vertical flip of the image provided by TA(left) and my own choice(right)


Fig.9, Fig.10 Both directional flip of the image provided by TA(left) and my own choice(right)

Rotation:

Rotation by an angle θ is expressed by the linear map

$$\begin{bmatrix} u_s \\ v_s \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} u_d \\ v_d \end{bmatrix},$$

where (u_d, v_d) is a destination location and (u_s, v_s) its corresponding coordinate in the original image. Because the mapped point generally falls between integer sample locations, the intensity is obtained by bilinear interpolation of its four nearest neighbors in the source grid. The output canvas is expanded to contain the rotated corners, which exposes background wedges near the perimeter; the rotated results for the two inputs are shown as Fig. 11 and Fig. 12.

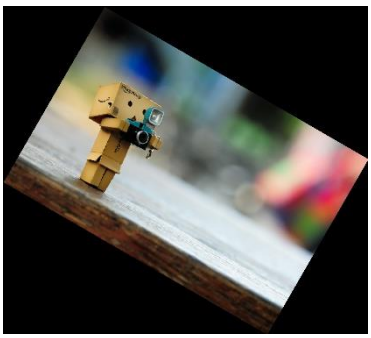


Fig 11, Fig.12 Rotation of $\pi/6$ rad of the image provided by TA(left) and my own choice(right)

Shearing:

Shearing is represented by the affine matrix

$$S = \begin{bmatrix} 1 & \text{shear}_x \\ \text{shear}_y & 1 \end{bmatrix}, \begin{bmatrix} u_s \\ v_s \end{bmatrix} = S^{-1} \begin{bmatrix} u_d \\ v_d \end{bmatrix}.$$

A horizontal shear ($\text{shear}_x \neq 0, \text{shear}_y = 0$) slants vertical structures into parallelograms while keeping horizontal lines parallel; a vertical shear ($\text{shear}_x = 0$, $\text{shear}_y \neq 0$) produces the complementary effect; general $(\text{shear}_x, \text{shear}_y)$ tilts along both axes. Since sampling locations are not generally on the integer lattice, pixel values are evaluated by bilinear interpolation at (u_s, v_s). In the tested setting, a purely horizontal shear is applied to the inputs, and the resulting images are presented as Fig. 13 and Fig. 14.

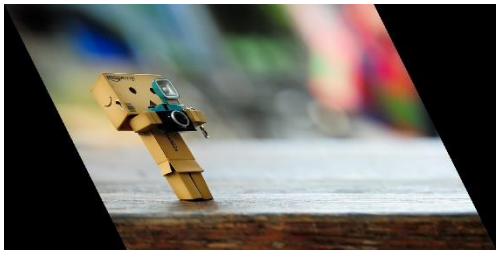

Fig.13, Fig.14 Shearing $(shear_x = 0.5)$ of the image provided by TA(left) and my own choice(right)

Scaling:

Resizing scales coordinates by factors $s_x, s_y > 0$ via $(u_d, v_d) = (s_x u_s, s_y v_s)$. The intensity at each destination point is obtained from the nearest source sample (nearest-neighbor rule), so magnification preserves discrete block boundaries while reduction suppresses high spatial frequencies and may drop fine details on repetitive

patterns. In this run, a scale of 1.5 and 0.6 is applied to the source images, but it is meaningless to attach the scaled images in the report, so no result will be shown here.

4. Discussion

This subsection addresses the question: what are the differences between forward warping and backward warping in image transformation, both from a subjective (visual/experiential) and an objective (mathematical/algorithmic) perspective?

Subjectively, forward warping tends to produce visible "holes" or gaps because source pixels are pushed to non-integer locations and may not land on every destination pixel; without additional splatting or blending, the result can look peppered or uneven, especially along thin lines and high-contrast edges. Backward warping, in contrast, feels visually coherent: each destination pixel actively looks up its value from the source, so the output canvas is uniformly filled and edges appear continuous rather than speckled. In our rotation example, the forward variant exhibits small unfilled spots unless explicit overlap-handling is added, whereas the backward variant yields a smooth, fully covered image with consistent backgrounds at the corners of the expanded canvas.

Objectively, the two methods differ in mapping direction and sampling mechanics. Forward warping computes $(u_d, v_d) = T(u_s, v_s)$ and transports each source sample to the destination; because (u_d, v_d) is rarely on the integer grid, one must distribute

each source value to nearby destination pixels (splatting) and reconcile overlaps where multiple sources map to the same destination. Backward warping instead inverts the map and evaluates $(u_s, v_s) = T^{-1}(u_d, v_d)$ for every destination pixel; since (u_s, v_s) is typically non-integer, the destination pulls a value via an interpolation rule (e.g., bilinear) or assigns a background if out of bounds. This inversion-centric view ensures complete coverage by construction and local, predictable memory access patterns. Computationally, forward warping requires careful accumulation and normalization to conserve intensity when footprints overlap, while backward warping requires only interpolation at a single source neighborhood per destination pixel. In practice, these distinctions explain why backward warping is the default for rotation, shear, and scaling in our lab: it guarantees a filled output and controlled resampling, whereas forward warping is primarily pedagogical here—to illustrate the coverage problem and the need for explicit splatting to mitigate holes.

Next, the question asks us to "use the forward warping method to implement image rotation, describe the flow, and show results, with code in LAB5/code/." I implemented this in rotation_forward.py, which is a true forward, source-driven rotator. The flow is: compute the rotation matrix and rotate the four image corners to obtain the new bounding box; from the min/max extents compute the output canvas size and the translation that moves all rotated coordinates into non-negative indices; traverse every

source pixel $(x_{\text{old}}, y_{\text{old}})$, compute its destination $(x_{\text{new}}, y_{\text{new}}) = R_\theta(x_{\text{old}}, y_{\text{old}}) + (\Delta_x, \Delta_y)$, round that destination to the nearest integer (the current code uses ceil), and "push" the source color into the output at that single destination location. This is a nearest-neighbor splat (no interpolation or accumulation), so multiple sources may land on the same output pixel while other output pixels receive no sample at all.

Compared with the backward rotation in rotation.py, the implementation differences are straightforward and explain the different look of the results. The backward version is destination-driven: for each output coordinate it applies the inverse rotation to find a sub-pixel location in the source image and then evaluates that value by bilinear interpolation of the four surrounding source samples. Because every output pixel is explicitly visited and assigned, the backward method fills the canvas densely and produces continuous edges with mild smoothing. The forward version visits sources instead, rounds destinations to integers, and writes a single pixel; this produces small "pinholes" where no source happened to land, occasional overlaps where several sources map to the same destination, and slight stair-stepping on thin edges due to the rounding choice.

The visual comparison bears this out. The forward-rotated outputs of our two inputs are the images denoted Fig. 15 and Fig. 16 in this report. They show the correct global re-orientation and canvas size, but you can see sparse unfilled pixels along high-contrast

contours and more jagged microstructure. The backward-rotated counterparts (shown earlier) look continuous and slightly smoother along edges, with the same geometric framing but without the lacunae characteristic of the forward splat.

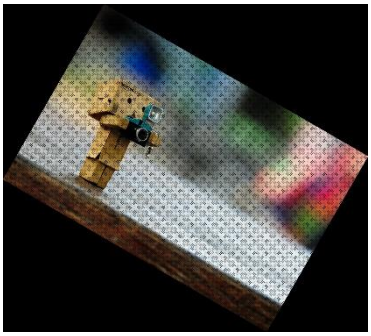


Fig 15, Fig.16 Forward rotation of the image provided by TA(left) and my own choice(right)

The last part of the discussion asks us to "implement at least one other method for gray scale, image flipping, image rotation, or image resize; specify the method, show the results, and compare with the original method, with code included in LAB5/code/." I addressed this by adding a second gray-scale conversion alongside the lab's BT.601 luminosity mapping. The original method computes luminance as $Y = 0.299R + 0.587G + 0.114B$ (kept three-channel for consistency). My alternative is the simple average-intensity mapping $Y_{\text{avg}} = (R + G + B)/3$, likewise replicated to three channels. Both implementations are placed in LAB5/code/grayscale.py (BT.601) and LAB5/code/grayscale_average.py (average).

Using the two lab source images, the BT.601 outputs are the gray results already shown earlier in the report (Fig. 3 and Fig. 4), while the new average-intensity results

are presented here as Fig. 17 and Fig. 18. The visual difference follows directly from the weighting: BT.601 emphasizes green and slightly down-weights blue and red, so midtones dominated by green (foliage, skin, many natural materials) appear a bit brighter than with the average; conversely, red- or blue-heavy regions can appear darker under BT.601 relative to the uniform average. To make the contrast precise, I also include pixel-wise difference visualizations between the two methods as Fig. 19 and Fig. 20 (white indicates larger magnitude differences). The bright contours around highlights and textured regions in these difference maps confirm that the average method compresses contrasts in green-rich areas while BT.601 preserves them more faithfully to human perception.


Fig.17, Fig.18 Averaged grayscale of the image provided by TA(left) and my own choice(right)


Fig.20, Fig.21 Grayscale difference of the image provided by TA(left) and my own choice(right)

In short, the original BT.601 method is perceptually weighted and better aligns with how luminance is perceived by the eye, producing images with more natural midtone balance and preserved detail in green-biased content. The proposed average method is simple and fast but discards the perceptual weighting; its results tend to be slightly flatter in scenes where one channel—especially green—dominates. Both codes are included under LAB5/code/, and the figures (BT.601: Fig. 3, Fig. 4; Average: Fig. 17, Fig. 18; Difference: Fig. 19, Fig. 20) document the qualitative distinctions clearly.

5. Conclusion

This lab built a concrete understanding of pixel-level geometric transformations by deriving the mapping equations and implementing them directly in NumPy. Using two common sources throughout allowed fair, side-by-side observation of how each operation alters structure and tone. Grayscale conversion confirmed the importance of perceptual weighting: BT.601 produced more natural mid-tone balance, while the alternative average-intensity method yielded flatter contrast in green-rich regions, as seen by comparing Fig. 3–4 with Fig. 17–18 and their difference maps in Fig. 19–20. Flipping behaved as exact isometries, preserving local detail while reversing orientation. Rotation and shear highlighted the role of resampling: the destination-driven implementation with inverse mapping and bilinear evaluation filled every pixel and maintained continuous edges, whereas the source-driven forward rotator, despite

correct geometry, exhibited sparse pinholes and jagged microstructure in Fig. 15–16 when only nearest-location splats were used. Scaling further emphasized the trade-offs between simplicity and fidelity: nearest-neighbor downsampling risks aliasing, while bilinear evaluation in the affine stages produced smoother, more visually coherent results.

Overall, the experiments show that mapping direction and sampling strategy determine visual quality as much as the transform itself. Backward mapping with interpolation is a robust default for affine warps because it guarantees coverage and predictable smoothness; forward mapping is pedagogically valuable and can match quality when augmented with proper splatting and normalization. If extended, the pipeline would benefit from area or bilinear splats for the forward case, anti-aliasing filters for minification, and a unified affine framework to compose transforms cleanly. These lessons—coordinate design, canvas reasoning, and resampling choices—provide a solid foundation for more advanced image registration and vision tasks.

6. Reference

[1] Department of Electrical Engineering, National Tsing Hua University, *EE3662-2025-Unit II-Visual Signal*.

[2] Department of Electrical Engineering, National Tsing Hua University, *Lab5*.